

3 л/р Асинхронное веб-приложение для отслеживания курсов валют

...

currency_tracker/

└─ app/

| └─ __init__.py

| └─ main.py

| └─ database.py

| └─ models.py

| └─ schemas.py

| └─ crud.py

| └─ routes/

| | └─ __init__.py

| | └─ users.py

| | └─ currencies.py

| | └─ subscriptions.py

| └─ services/

| └─ __init__.py

| └─ cbr_service.py

└─ requirements.txt

└─ .env

...

1. Установка зависимостей

****requirements.txt:****

```
```txt
```

```
fastapi==0.104.1
```

```
uvicorn[standard]==0.24.0
```

```
sqlalchemy==2.0.23
```

```
aiosqlite==0.19.0
```

```
httpx==0.25.0
```

```
python-dotenv==1.0.0
```

```
pydantic==2.5.0
```

```
pydantic-settings==2.1.0
```

```
```
```

2. Конфигурация и база данных

```
**app/database.py:**
```

```
```python
```

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker
```

```
from sqlalchemy.orm import declarative_base
```

```
from typing import AsyncGenerator
```

```
import os
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite+aiosqlite:///./currency_tracker.db")
```

```
engine = create_async_engine(
```

```
 DATABASE_URL,
```

```
 echo=True,
```

```
 future=True,
```

```
)
```

```
AsyncSessionLocal = async_sessionmaker(
 engine,
 class_=AsyncSession,
 expire_on_commit=False,
)
```

```
Base = declarative_base()
```

```
async def get_db() -> AsyncGenerator[AsyncSession, None]:
 async with AsyncSessionLocal() as session:
 try:
 yield session
 finally:
 await session.close()
```

```
async def init_db():
 async with engine.begin() as conn:
 await conn.run_sync(Base.metadata.create_all)
 ...
```

## 3. Модели данных

```
app/models.py:
```

```
```python
```

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey, Table, Float
```

```
from sqlalchemy.orm import relationship
```

```
from sqlalchemy.sql import func
```

```
from app.database import Base
```

```
# Таблица для связи many-to-many
```

```
subscription = Table(
    'subscription',
    Base.metadata,
    Column('user_id', Integer, ForeignKey('users.id', ondelete='CASCADE'), primary_key=True),
    Column('currency_id', Integer, ForeignKey('currencies.id', ondelete='CASCADE'),
primary_key=True),
)
```

```
class User(Base):
```

```
    __tablename__ = 'users'
```

```
    id = Column(Integer, primary_key=True, index=True)
```

```
    username = Column(String, unique=True, nullable=False, index=True)
```

```
    email = Column(String, unique=True, nullable=False, index=True)
```

```
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

```
# Связь с валютами через подписки
```

```
currencies = relationship(
    "Currency",
    secondary=subscription,
    back_populates="users",
    lazy="selectin"
)
```

```
class Currency(Base):
```

```
    __tablename__ = 'currencies'
```

```

id = Column(Integer, primary_key=True, index=True)
code = Column(String, unique=True, nullable=False, index=True)
name = Column(String, nullable=False)
rate = Column(Float, nullable=True)
nominal = Column(Integer, default=1)
updated_at = Column(DateTime(timezone=True), server_default=func.now(),
onupdate=func.now())

```

Связь с пользователями через подписки

```

users = relationship(
    "User",
    secondary=subscription,
    back_populates="currencies",
    lazy="selectin"
)
'''

```

4. Pydantic схемы

****app/schemas.py:****

```

```python

```

```

from pydantic import BaseModel, EmailStr, Field, validator

```

```

from typing import Optional, List

```

```

from datetime import datetime

```

```

class UserBase(BaseModel):

```

```

 username: str = Field(..., min_length=3, max_length=50)

```

```

 email: EmailStr

```

```
class UserCreate(UserBase):
```

```
 pass
```

```
class UserUpdate(BaseModel):
```

```
 username: Optional[str] = Field(None, min_length=3, max_length=50)
```

```
 email: Optional[EmailStr] = None
```

```
class CurrencyBase(BaseModel):
```

```
 code: str = Field(..., min_length=3, max_length=10)
```

```
 name: str
```

```
 nominal: int = 1
```

```
 rate: Optional[float] = None
```

```
class CurrencyResponse(CurrencyBase):
```

```
 id: int
```

```
 updated_at: Optional[datetime] = None
```

```
class Config:
```

```
 from_attributes = True
```

```
class UserResponse(UserBase):
```

```
 id: int
```

```
 created_at: Optional[datetime] = None
```

```
 currencies: List[CurrencyResponse] = []
```

```
class Config:
```

```
 from_attributes = True
```

```
class SubscriptionCreate(BaseModel):
```

```
 user_id: int
```

```
 currency_code: str
```

```
class SubscriptionDelete(BaseModel):
```

```
 user_id: int
```

```
 currency_code: str
```

```
class CurrencyRateResponse(BaseModel):
```

```
 code: str
```

```
 name: str
```

```
 rate: float
```

```
 nominal: int
```

```
 updated_at: datetime
```

```
``
```

```
5. CRUD операции
```

```
app/crud.py:
```

```
```python
```

```
from sqlalchemy.ext.asyncio import AsyncSession
```

```
from sqlalchemy import select, delete, and_
```

```
from sqlalchemy.exc import IntegrityError
```

```
from typing import Optional, List
```

```
from app import models, schemas
```

```
# User CRUD
```

```
async def get_user(db: AsyncSession, user_id: int) -> Optional[models.User]:
```

```
    result = await db.execute(
```

```
        select(models.User).where(models.User.id == user_id)
    )
    return result.scalar_one_or_none()
```

```
async def get_user_by_username(db: AsyncSession, username: str) -> Optional[models.User]:
    result = await db.execute(
        select(models.User).where(models.User.username == username)
    )
    return result.scalar_one_or_none()
```

```
async def get_user_by_email(db: AsyncSession, email: str) -> Optional[models.User]:
    result = await db.execute(
        select(models.User).where(models.User.email == email)
    )
    return result.scalar_one_or_none()
```

```
async def get_users(db: AsyncSession, skip: int = 0, limit: int = 100) -> List[models.User]:
    result = await db.execute(
        select(models.User).offset(skip).limit(limit)
    )
    return result.scalars().all()
```

```
async def create_user(db: AsyncSession, user: schemas.UserCreate) -> models.User:
    db_user = models.User(
        username=user.username,
        email=user.email
    )
    db.add(db_user)
    await db.commit()
```



```
await db.refresh(db_user)

return db_user
```

```
async def update_user(
    db: AsyncSession,
    user_id: int,
    user_update: schemas.UserUpdate
) -> Optional[models.User]:
    db_user = await get_user(db, user_id)
    if not db_user:
        return None

    update_data = user_update.model_dump(exclude_unset=True)
    for key, value in update_data.items():
        setattr(db_user, key, value)

    await db.commit()
    await db.refresh(db_user)
    return db_user
```

```
async def delete_user(db: AsyncSession, user_id: int) -> bool:
    db_user = await get_user(db, user_id)
    if not db_user:
        return False

    await db.delete(db_user)
    await db.commit()
    return True
```

Currency CRUD

```
async def get_currency(db: AsyncSession, currency_id: int) -> Optional[models.Currency]:
```

```
    result = await db.execute(
        select(models.Currency).where(models.Currency.id == currency_id)
    )
    return result.scalar_one_or_none()
```

```
async def get_currency_by_code(db: AsyncSession, code: str) -> Optional[models.Currency]:
```

```
    result = await db.execute(
        select(models.Currency).where(models.Currency.code == code)
    )
    return result.scalar_one_or_none()
```

```
async def get_currencies(db: AsyncSession) -> List[models.Currency]:
```

```
    result = await db.execute(select(models.Currency))
    return result.scalars().all()
```

```
async def create_or_update_currency(
```

```
    db: AsyncSession,
    code: str,
    name: str,
    rate: float = None,
    nominal: int = 1
```

```
) -> models.Currency:
```

```
    currency = await get_currency_by_code(db, code)
    if currency:
        currency.name = name
        if rate is not None:
            currency.rate = rate
```

```
        currency.nominal = nominal
```

```
    else:
```

```
        currency = models.Currency(
```

```
            code=code,
```

```
            name=name,
```

```
            rate=rate,
```

```
            nominal=nominal
```

```
        )
```

```
        db.add(currency)
```

```
    await db.commit()
```

```
    await db.refresh(currency)
```

```
    return currency
```

```
# Subscription CRUD
```

```
async def create_subscription(
```

```
    db: AsyncSession,
```

```
    user_id: int,
```

```
    currency_code: str
```

```
) -> Optional[models.User]:
```

```
    user = await get_user(db, user_id)
```

```
    if not user:
```

```
        return None
```

```
    currency = await get_currency_by_code(db, currency_code)
```

```
    if not currency:
```

```
        return None
```

```
# Проверка на существующую подписку
```

```
if currency in user.currencies:
```

```
    return None
```

```
user.currencies.append(currency)
```

```
await db.commit()
```

```
await db.refresh(user)
```

```
return user
```

```
async def delete_subscription(
```

```
    db: AsyncSession,
```

```
    user_id: int,
```

```
    currency_code: str
```

```
) -> bool:
```

```
    user = await get_user(db, user_id)
```

```
    if not user:
```

```
        return False
```

```
    currency = await get_currency_by_code(db, currency_code)
```

```
    if not currency:
```

```
        return False
```

```
    if currency not in user.currencies:
```

```
        return False
```

```
    user.currencies.remove(currency)
```

```
    await db.commit()
```

```
    return True
```

```
async def get_user_subscriptions(db: AsyncSession, user_id: int) -> List[models.Currency]:
```

```

user = await get_user(db, user_id)

if not user:

    return []

return user.currencies
...

```

6. Сервис для работы с API ЦБ РФ

```

**app/services/cbr_service.py:**

``python

import httpx

import xml.etree.ElementTree as ET

from typing import List, Dict, Tuple

from datetime import datetime

class CBRService:

    CBR_URL = "https://www.cbr.ru/scripts/XML_daily.asp"

    @staticmethod

    async def get_currencies() -> List[Dict[str, any]]:

        """

        Получение курсов валют от ЦБ РФ

        """

        async with httpx.AsyncClient() as client:

            response = await client.get(CBRService.CBR_URL)

            response.raise_for_status()

            return CBRService._parse_xml(response.text)

```

```
@staticmethod
```

```
def _parse_xml(xml_data: str) -> List[Dict[str, any]]:
```

```
    """
```

```
    Парсинг XML ответа от ЦБ РФ
```

```
    """
```

```
    root = ET.fromstring(xml_data)
```

```
    currencies = []
```

```
    for valute in root.findall('Valute'):
```

```
        code = valute.find('CharCode').text
```

```
        name = valute.find('Name').text
```

```
        value = valute.find('Value').text.replace(',', '.')
```

```
        nominal = int(valute.find('Nominal').text)
```

```
        currencies.append({
```

```
            'code': code,
```

```
            'name': name,
```

```
            'rate': float(value) / nominal,
```

```
            'nominal': nominal
```

```
        })
```

```
    return currencies
```

```
...
```

```
## 7. Роуты пользователей
```

```
**app/routes/users.py:**
```

```
```python
```

```
from fastapi import APIRouter, Depends, HTTPException, status
```

```

from sqlalchemy.ext.asyncio import AsyncSession

from typing import List

from app import crud, schemas

from app.database import get_db

router = APIRouter(prefix="/users", tags=["users"])

@router.post("/", response_model=schemas.UserResponse,
status_code=status.HTTP_201_CREATED)
async def create_user(
 user: schemas.UserCreate,
 db: AsyncSession = Depends(get_db)
):
 # Проверка на существование пользователя
 existing_user = await crud.get_user_by_username(db, user.username)
 if existing_user:
 raise HTTPException(
 status_code=status.HTTP_400_BAD_REQUEST,
 detail="Username already registered"
)

 existing_email = await crud.get_user_by_email(db, user.email)
 if existing_email:
 raise HTTPException(
 status_code=status.HTTP_400_BAD_REQUEST,
 detail="Email already registered"
)

 return await crud.create_user(db, user)

```

```
@router.get("/", response_model=List[schemas.UserResponse])
```

```
async def get_users(
```

```
 skip: int = 0,
```

```
 limit: int = 100,
```

```
 db: AsyncSession = Depends(get_db)
```

```
):
```

```
 return await crud.get_users(db, skip=skip, limit=limit)
```

```
@router.get("/{user_id}", response_model=schemas.UserResponse)
```

```
async def get_user(
```

```
 user_id: int,
```

```
 db: AsyncSession = Depends(get_db)
```

```
):
```

```
 user = await crud.get_user(db, user_id)
```

```
 if not user:
```

```
 raise HTTPException(
```

```
 status_code=status.HTTP_404_NOT_FOUND,
```

```
 detail="User not found"
```

```
)
```

```
 return user
```

```
@router.put("/{user_id}", response_model=schemas.UserResponse)
```

```
async def update_user(
```

```
 user_id: int,
```

```
 user_update: schemas.UserUpdate,
```

```
 db: AsyncSession = Depends(get_db)
```

```
):
```

```
 # Проверка на существование пользователя
```



```
existing_user = await crud.get_user(db, user_id)
```

```
if not existing_user:
```

```
 raise HTTPException(
 status_code=status.HTTP_404_NOT_FOUND,
 detail="User not found"
)
```

```
Проверка уникальности username и email
```

```
if user_update.username:
```

```
 user_with_username = await crud.get_user_by_username(db, user_update.username)
```

```
 if user_with_username and user_with_username.id != user_id:
```

```
 raise HTTPException(
 status_code=status.HTTP_400_BAD_REQUEST,
 detail="Username already taken"
)
```

```
if user_update.email:
```

```
 user_with_email = await crud.get_user_by_email(db, user_update.email)
```

```
 if user_with_email and user_with_email.id != user_id:
```

```
 raise HTTPException(
 status_code=status.HTTP_400_BAD_REQUEST,
 detail="Email already registered"
)
```

```
updated_user = await crud.update_user(db, user_id, user_update)
```

```
return updated_user
```

```
@router.delete("/{user_id}", status_code=status.HTTP_204_NO_CONTENT)
```

```
async def delete_user(
```

```

 user_id: int,

 db: AsyncSession = Depends(get_db)
):
 deleted = await crud.delete_user(db, user_id)

 if not deleted:
 raise HTTPException(
 status_code=status.HTTP_404_NOT_FOUND,
 detail="User not found"
)

 return None
 ...

```

## ## 8. Роуты валют

```

app/routes/currencies.py:

``python
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.ext.asyncio import AsyncSession
from typing import List
from app import crud, schemas
from app.database import get_db
from app.services.cbr_service import CBRService

router = APIRouter(prefix="/currencies", tags=["currencies"])

@router.get("/", response_model=List[schemas.CurrencyResponse])
async def get_currencies(
 db: AsyncSession = Depends(get_db)
):

```

```
return await crud.get_currencies(db)
```

```
@router.post("/update", response_model=List[schemas.CurrencyResponse])
```

```
async def update_currencies(
```

```
 db: AsyncSession = Depends(get_db)
```

```
):
```

```
 try:
```

```
 currencies_data = await CBRService.get_currencies()
```

```
 updated_currencies = []
```

```
 for currency_data in currencies_data:
```

```
 currency = await crud.create_or_update_currency(
```

```
 db,
```

```
 code=currency_data['code'],
```

```
 name=currency_data['name'],
```

```
 rate=currency_data['rate'],
```

```
 nominal=currency_data['nominal']
```

```
)
```

```
 updated_currencies.append(currency)
```

```
 return updated_currencies
```

```
 except Exception as e:
```

```
 raise HTTPException(
```

```
 status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
```

```
 detail=f"Failed to update currencies from CBR: {str(e)}"
```

```
)
```

```
@router.get("/{currency_code}/rate", response_model=schemas.CurrencyRateResponse)
```

```
async def get_currency_rate(
```

```

currency_code: str,

db: AsyncSession = Depends(get_db)
):

currency = await crud.get_currency_by_code(db, currency_code.upper())

if not currency:

 raise HTTPException(

 status_code=status.HTTP_404_NOT_FOUND,

 detail="Currency not found"

)

if currency.rate is None:

 raise HTTPException(

 status_code=status.HTTP_404_NOT_FOUND,

 detail="Rate not available for this currency"

)

return schemas.CurrencyRateResponse(

 code=currency.code,

 name=currency.name,

 rate=currency.rate,

 nominal=currency.nominal,

 updated_at=currency.updated_at

)
...

```

## ## 9. Роуты подписок

```

app/routes/subscriptions.py:

```python

```

```

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.ext.asyncio import AsyncSession
from app import crud, schemas
from app.database import get_db

router = APIRouter(prefix="/subscriptions", tags=["subscriptions"])

@router.post("/", response_model=schemas.UserResponse,
status_code=status.HTTP_201_CREATED)
async def create_subscription(
    subscription: schemas.SubscriptionCreate,
    db: AsyncSession = Depends(get_db)
):
    user = await crud.create_subscription(
        db,
        user_id=subscription.user_id,
        currency_code=subscription.currency_code.upper()
    )

    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="User or currency not found"
        )

    # Проверка на существующую подписку через количество валют
    # Если количество не изменилось - подписка уже существовала
    return user

```

```

@router.delete("/", status_code=status.HTTP_204_NO_CONTENT)
async def delete_subscription(
    subscription: schemas.SubscriptionDelete,
    db: AsyncSession = Depends(get_db)
):
    deleted = await crud.delete_subscription(
        db,
        user_id=subscription.user_id,
        currency_code=subscription.currency_code.upper()
    )

    if not deleted:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Subscription not found or user/currency does not exist"
        )

    return None
...

```

10. Главное приложение

```

**app/main.py:**

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
from app.database import init_db
from app.routes import users, currencies, subscriptions

```

```
@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
```

```
 # Запуск при инициализации
```

```
 await init_db()
```

```
 yield
```

```
 # Закрытие при завершении
```

```
app = FastAPI(
```

```
 title="Currency Tracker API",
```

```
 description="Асинхронное веб-приложение для отслеживания курсов валют",
```

```
 version="1.0.0",
```

```
 lifespan=lifespan
```

```
)
```

```
CORS настройки
```

```
app.add_middleware(
```

```
 CORSMiddleware,
```

```
 allow_origins=["*"],
```

```
 allow_credentials=True,
```

```
 allow_methods=["*"],
```

```
 allow_headers=["*"],
```

```
)
```

```
Подключение роутеров
```

```
app.include_router(users.router)
```

```
app.include_router(currencies.router)
```

```
app.include_router(subscriptions.router)
```

```
@app.get("/")
async def root():
 return {
 "message": "Currency Tracker API",
 "docs": "/docs",
 "redoc": "/redoc"
 }
```

```
if __name__ == "__main__":
 import uvicorn
 uvicorn.run(
 "app.main:app",
 host="0.0.0.0",
 port=8000,
 reload=True
)
...
```

## ## 11. Запуск приложения

**\*\*Команды для запуска:\*\***

```
```bash
```

```
# Установка зависимостей
```

```
pip install -r requirements.txt
```

```
# Запуск приложения
```

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```


Или через Python

```
python -m app.main
```

```
...
```

****env файл:****

```
...
```

```
DATABASE_URL=sqlite+aiosqlite:///./currency_tracker.db
```

```
...
```

12. Тестирование API

****Примеры запросов с использованием curl:****

```
```bash
```

# Создание пользователя

```
curl -X POST "http://localhost:8000/users/" \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"username": "john_doe", "email": "john@example.com"}'
```

# Получение всех пользователей

```
curl "http://localhost:8000/users/"
```

# Получение пользователя по ID

```
curl "http://localhost:8000/users/1"
```

# Обновление пользователя

```
curl -X PUT "http://localhost:8000/users/1" \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"email": "john_new@example.com"}'
```

# Обновление курсов валют

```
curl -X POST "http://localhost:8000/currencies/update"
```

# Получение всех валют

```
curl "http://localhost:8000/currencies/"
```

# Получение курса конкретной валюты

```
curl "http://localhost:8000/currencies/USD/rate"
```

# Подписка на валюту

```
curl -X POST "http://localhost:8000/subscriptions/" \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"user_id": 1, "currency_code": "USD"}'
```

# Отписка от валюты

```
curl -X DELETE "http://localhost:8000/subscriptions/" \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"user_id": 1, "currency_code": "USD"}'
```

# Удаление пользователя

```
curl -X DELETE "http://localhost:8000/users/1"
```

```
...
```

## 13. Дополнительные улучшения

**\*\*app/middleware.py (логирование запросов):\*\***

```
```python
```

```
from fastapi import Request
```

```

from starlette.middleware.base import BaseHTTPMiddleware

import logging

import time

logger = logging.getLogger(__name__)

class LoggingMiddleware(BaseHTTPMiddleware):

    async def dispatch(self, request: Request, call_next):

        start_time = time.time()

        response = await call_next(request)

        process_time = time.time() - start_time

        logger.info(

            f"Method: {request.method} Path: {request.url.path} "

            f"Status: {response.status_code} Time: {process_time:.3f}s"

        )

        return response
...

```

Это полноценное асинхронное веб-приложение, которое полностью соответствует всем требованиям технического задания. Оно предоставляет REST API для управления пользователями, валютами и подписками с интеграцией с API Центрального банка РФ.